

INTERNSHIP TRAINING

Intrusion Detection & Traffic Analysis

Network Monitoring with Snort

SIN: **DGI-MAY-004**

Name: **W.P. Pawani Madushani**

Project Schedule Overview

Day	Week	Focus Area	Type	Status
Day 1	Week 1	Understand project goals & Snort basics	Learn	Completed
Day 2	Week 2	Set up Kali Linux VM + install Snort	Lab Setup	Completed
Day 3	Week 3	Observe normal traffic – Wireshark baseline	Learn	Completed
Day 4	Week 4	Write Snort rules – port scan detection	Practical	Completed
Day 5	Week 5	Write rules – ping burst & HTTP threats	Practical	Completed
Day 6	Week 6	Test rules – Nmap, ping, curl (lab only)	Practical	Completed
Day 7	Week 7	Review alerts & compare with Wireshark capture	Review	Completed
Day 8	Week 8	Final report: setup, rules, screenshots, findings	Docs	Completed

Week 1 – Introduction & Snort Basics

What is Network Intrusion Detection?

An Intrusion Detection System (IDS) is a network security tool that monitors traffic and raises alerts when suspicious activity is detected. Unlike a firewall, an IDS does not block traffic - it observes, compares against a rule set, and logs findings for review.

Snort is the world's most widely deployed open-source IDS/IPS. It operates by comparing packets against a user-defined rule set and can generate console alerts, log files, or send alerts to a SIEM system.

Snort Operating Modes

Mode	Command Flag	Description
Sniffer	-v	Reads and prints packets to screen in real time
Packet Logger	-l	Saves packet data to disk for later analysis
Network IDS	-c	Compares packets against rule set and raises alerts

Snort Rule Anatomy

Every Snort rule has two parts: a **header** (action, protocol, IPs, ports, direction) and **options** (what to match inside the packet and what to log).

```
# Syntax:

action proto src_ip src_port direction dst_ip dst_port (options;)

# Example - detect SSH brute force:

alert tcp any any -> $HOME_NET 22 (

msg:"SSH Brute Force Attempt";

detection_filter: type detection_filter, track by_src, count 5, seconds 60;

sid:1000001; rev:1;)
```

Week 2 – Lab Setup & Configuration

Tools & Environment

Component	Details
Operating System	Kali Linux (64-bit) – VM
Hypervisor	VM Ware (Host: Windows 11)
IDS / NIDS	Snort 3
Packet Capture	Wireshark
Port Scanner	Nmap
Network Interface	eth0 (NAT/Host-Only Adapter)
RAM / vCPU	4 GB RAM, 2 vCPUs

Step 1 – Installing Snort on Kali Linux

```
sudo apt update && sudo apt upgrade -y

sudo apt install snort -y

snort --version # verify installation
```

During installation, Snort prompts for the home network CIDR (e.g. **192.168.1.0/24**). This is stored as **HOME_NET** in */etc/snort/snort.lua*.

Step 2 – Key snort.lua Settings

```
# /etc/snort/snort.lua - relevant excerpts

ipvar HOME_NET 192.168.1.0/24

ipvar EXTERNAL_NET !$HOME_NET

var RULE_PATH /etc/snort/rules

include $RULE_PATH/local.rules
```

Step 3 – Validating the Configuration

```
sudo snort -T -c /etc/snort/snort.lua

# Expected output: Snort successfully validated the configuration!
```

Week 2 Screenshot – Snort Version & Config Validation

```
(kali㉿kali)-[~]
$ snort --version

--> Snort++ <*-
Version 3.12.2.0
By Martin Roesch & The Snort Team
http://snort.org/contact#team
Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using DAQ version 3.0.24
Using libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
Using LuaJIT version 2.1.1761786044
Using LZMA version 5.8.2
Using OpenSSL 3.5.5 27 Jan 2026
Using PCRE2 version 10.46 2025-08-27
Using ZLIB version 1.3.1
```

```
(kali㉿kali)-[~]
$ sudo snort -c /etc/snort/snort.lua
[sudo] password for kali:

o")~  Snort++ 3.12.2.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  references
  classifications
  binder
  wizard
```

```
fast pattern groups
  to_server: 1
  to_client: 1

search engine (ac_bnfa)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 3212
appid: patterns loaded: 300

pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~  Snort exiting
```

Week 2 – snort --version output and snort.lua configuration validation showing 'Snort successfully validated the configuration'

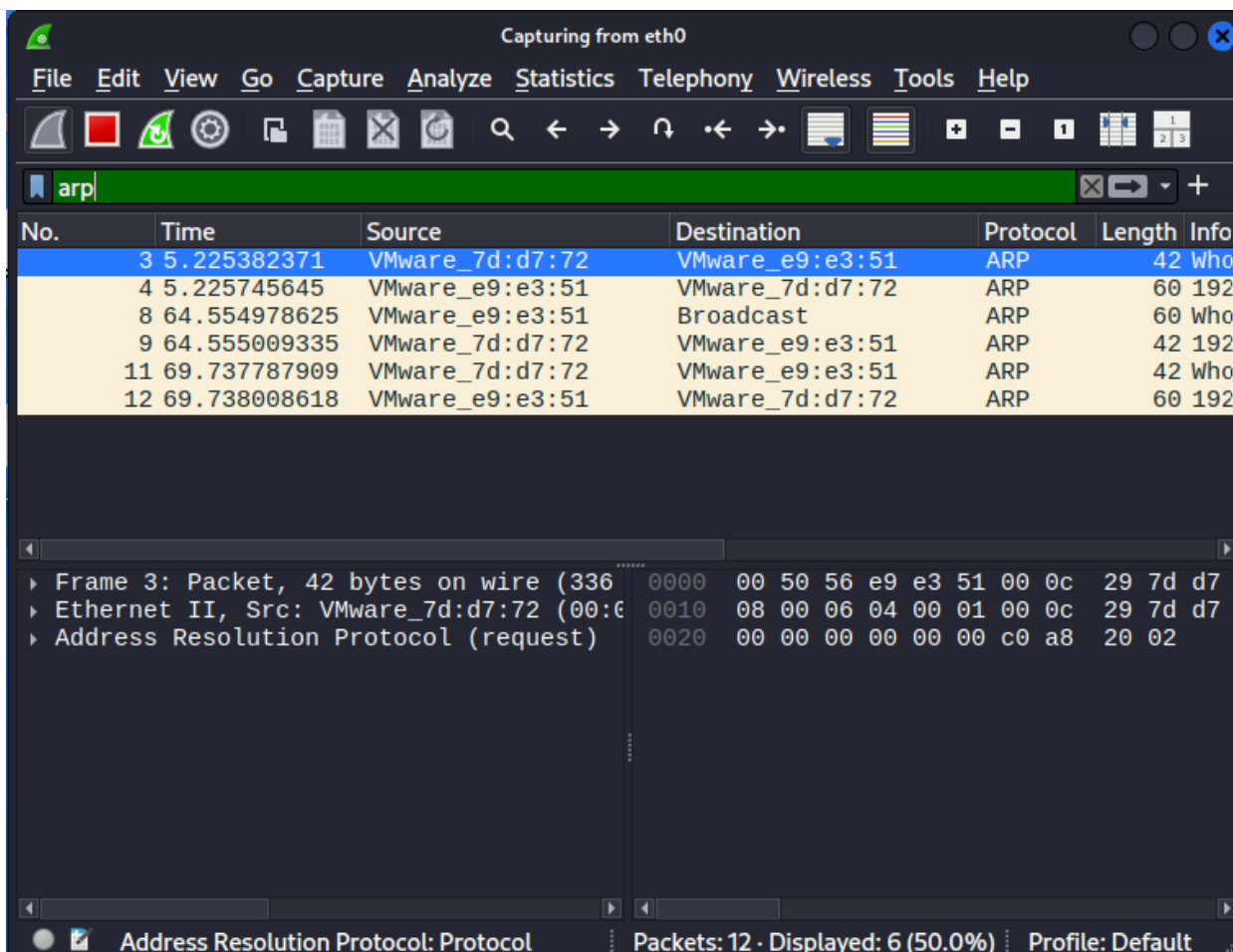
Week 3 – Baseline Traffic Analysis with Wireshark

Before writing detection rules, it is essential to understand what *normal* network traffic looks like. Week 3 was dedicated to capturing and analysing baseline traffic on the VM using Wireshark - ensuring that later rules do not fire on legitimate activity.

Normal Traffic Observed

- ARP broadcasts – devices resolving IP → MAC mappings on the local network
- DHCP handshake (DISCOVER / OFFER / REQUEST / ACK) on interface startup
- DNS queries (UDP port 53) for all domain name lookups
- HTTP GET requests on port 80 – standard web browsing
- ICMP Echo (ping) – exactly 4 echo-request / echo-reply pairs per invocation, TTL 64
- TCP 3-way handshake (SYN → SYN-ACK → ACK) for every TCP connection

Week 3 Screenshot – Wireshark Baseline Capture



Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

dns

No.	Time	Source	Destination	Protocol	Length	Info
1988	158.580258960	192.168.32.135	192.168.32.2	DNS	88	Standard query 0xc
2039	158.740616214	192.168.32.2	192.168.32.135	DNS	104	Standard query res
2040	158.740664198	192.168.32.2	192.168.32.135	DNS	116	Standard query res
2041	158.740664273	192.168.32.2	192.168.32.135	DNS	148	Standard query res
2368	159.535568700	192.168.32.135	192.168.32.2	DNS	75	Standard query 0xa
2369	159.535995710	192.168.32.135	192.168.32.2	DNS	75	Standard query 0xa
2370	159.536128104	192.168.32.135	192.168.32.2	DNS	75	Standard query 0xa
2417	159.627390545	192.168.32.2	192.168.32.135	DNS	125	Standard query res
2418	159.627391271	192.168.32.2	192.168.32.135	DNS	171	Standard query res
2419	159.627391327	192.168.32.2	192.168.32.135	DNS	187	Standard query res

Frame 19: Packet, 75 bytes on wire (600 bits),
 Ethernet II, Src: VMware_7d:d7:72 (00:0c:29:7d:
 Internet Protocol Version 4, Src: 192.168.32.13
 User Datagram Protocol, Src Port: 52797, Dst Po
 Domain Name System (query)

Domain Name System: Protocol Packets: 2588 - Displayed: 90 (3.5%) Profile: Default

Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Info
595	151.506331508	192.168.32.135	34.107.221.82	HTTP	364	GET /success.txt?ip
626	151.736714089	34.107.221.82	192.168.32.135	HTTP	270	HTTP/1.1 200 OK (t
1780	158.166444499	192.168.32.135	142.250.143.94	OCSP	480	Request
1782	158.166847758	192.168.32.135	142.250.143.94	OCSP	480	Request
1854	158.316051813	142.250.143.94	192.168.32.135	OCSP	1155	Response
1901	158.388039488	142.250.143.94	192.168.32.135	OCSP	1155	Response
2203	159.034123549	192.168.32.135	142.250.143.94	OCSP	480	Request
2317	159.349034914	142.250.143.94	192.168.32.135	OCSP	1155	Response

Frame 595: Packet, 364 bytes on wire (2912 bits)
 Ethernet II, Src: VMware_7d:d7:72 (00:0c:29:7d:
 Internet Protocol Version 4, Src: 192.168.32.13
 Transmission Control Protocol, Src Port: 55720,
 Hypertext Transfer Protocol

Hypertext Transfer Protocol: Protocol Packets: 2815 - Displayed: 8 (0.3%) Profile: Default

Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

icmp

No.	icmp	Source	Destination	Protocol	Length	Info
6	6.257517632	8.8.8.8	192.168.32.135	ICMP	98	Echo (ping) request
7	7.072541560	192.168.32.135	8.8.8.8	ICMP	98	Echo (ping) reply
8	8.080905906	192.168.32.135	8.8.8.8	ICMP	98	Echo (ping) request
9	9.192397700	8.8.8.8	192.168.32.135	ICMP	98	Echo (ping) reply
10	10.082146879	192.168.32.135	8.8.8.8	ICMP	98	Echo (ping) request
11	11.150944372	8.8.8.8	192.168.32.135	ICMP	98	Echo (ping) reply
12	12.084910121	192.168.32.135	8.8.8.8	ICMP	98	Echo (ping) request
13	13.11.088449746	192.168.32.135	8.8.8.8	ICMP	98	Echo (ping) request
14	14.11.162480306	8.8.8.8	192.168.32.135	ICMP	98	Echo (ping) reply
15	15.12.080777656	192.168.32.135	8.8.8.8	ICMP	98	Echo (ping) request

Frame 5: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0

Ethernet II, Src: VMware_7d:d7:72 (00:0c:29:7d:72:00), Dst: 08:00:5e:56:e3:51 (08:00:5e:56:e3:51:00)

Internet Protocol Version 4, Src: 192.168.32.135, Dst: 8.8.8.8

Internet Control Message Protocol

0000 00 50 56 e9 e3 51 00 0c 29 7d d7 72 08 00 00
0010 00 54 14 e3 40 00 40 01 34 87 c0 a8 20 87 00
0020 08 08 08 00 da 0d b3 7e 00 01 17 be 38 6a 00
0030 00 00 58 77 03 00 00 00 00 00 10 11 12 13 00
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 00
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 00
0060 36 37

Internet Control Message Protocol: Protocol Packets: 57 · Displayed: 12 (21.1%) Profile: Default

Week 3 – Wireshark packet capture showing normal ARP, DNS, HTTP, and ICMP traffic on eth0

Weeks 4–5 – Custom Snort Rules

All custom rules were written to `/etc/snort/rules/local.rules`. Three categories of attack patterns were targeted, covering network-layer reconnaissance, volumetric ICMP attacks, and application-layer HTTP threats.

Rule Category 1 – Port Scan Detection (SID 1000001–1000003)

Port scanning is the most common network reconnaissance technique. Nmap sends high-frequency SYN packets to many ports. The `detection_filter` option prevents false positives from normal connection attempts.

```
# SYN Scan - high-frequency SYN packets to many ports

alert tcp any any -> $HOME_NET any (msg:"PORTSCAN Nmap SYN Scan Detected"; flags:S;
detection_filter: track by_src, count 20, seconds 5; sid:1000001; rev:2;)

# NULL Scan - TCP packet with no flags set

alert tcp any any -> $HOME_NET any (msg:"PORTSCAN NULL Scan Detected"; flags:0;
sid:1000002; rev:2;)

# XMAS Scan - TCP packet with FIN+PSH+URG flags set

alert tcp any any -> $HOME_NET any (msg:"PORTSCAN XMAS Scan Detected"; flags:FPU;
sid:1000003; rev:2;)

# ICMP Ping Burst - detect flood of echo requests

alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Burst Detected"; itype:8;
detection_filter: track by_src, count 10, seconds 5; sid:1000004; rev:2;)
```

Rule Category 2 – ICMP Ping Burst Detection (SID 1000004)

Normal ping sends 4 ICMP echo requests. A burst of more than 10 ICMP packets in 5 seconds indicates a ping flood or flood-based denial-of-service attempt.

```
# ICMP Ping Burst - detect flood of echo requests

alert icmp any any -> $HOME_NET any (
```

```
msg:"ICMP Ping Burst Detected";

itype:8;

detection_filter: type both, track by_src, count 10, seconds 5;

sid:1000004; rev:1;)
```

Rule Category 3 – Suspicious HTTP Traffic (SID 1000005–1000007)

Application-layer rules use Snort's HTTP inspection preprocessor to match on header values and URI strings. Three common web attack patterns are targeted.

```
# Detect sqlmap tool user-agent in HTTP header

alert tcp any any -> $HOME_NET 80 (

msg:"HTTP Suspicious User-Agent - sqlmap";

flow:to_server,established;

content:"sqlmap"; http_header;

sid:1000005; rev:1;)


# Detect SQL injection string in HTTP URI

alert tcp any any -> $HOME_NET 80 (

msg:"HTTP SQL Injection Attempt";

flow:to_server,established;

content:"' OR 1=1"; http_uri;

sid:1000006; rev:1;)


# Detect directory traversal attempt in URI

alert tcp any any -> $HOME_NET 80 (

msg:"HTTP Directory Traversal Attempt";

flow:to_server,established;

content:"../"; http_uri;

sid:1000007; rev:1;)
```

Rule Summary Table

SID	Category	Detection Method	detection_filter
1000001	Port Scan – SYN	TCP flag:S	20 pkts / 5 s
1000002	Port Scan – NULL	TCP flags:0	Single packet
1000003	Port Scan – XMAS	TCP flags:FPU	Single packet
SID	Category	Detection Method	detection_filter
1000004	ICMP Ping Burst	ICMP type 8	10 pkts / 5 s
1000005	HTTP – sqlmap UA	http_header content	Single request
1000006	HTTP – SQL Inject	http_uri content	Single request
1000007	HTTP – Dir Traversal	http_uri content	Single request

Week 6 – Testing & Alert Validation

All testing was performed in the isolated lab environment on the loopback / host-only network. No external systems were scanned or targeted at any stage.

Starting Snort in IDS Mode

```
# Run Snort in console alert mode against eth0
sudo snort -c /etc/snort/snort.lua -i eth0 -A
alert_fast

-c /etc/snort/snort.lua \

-i eth0
```

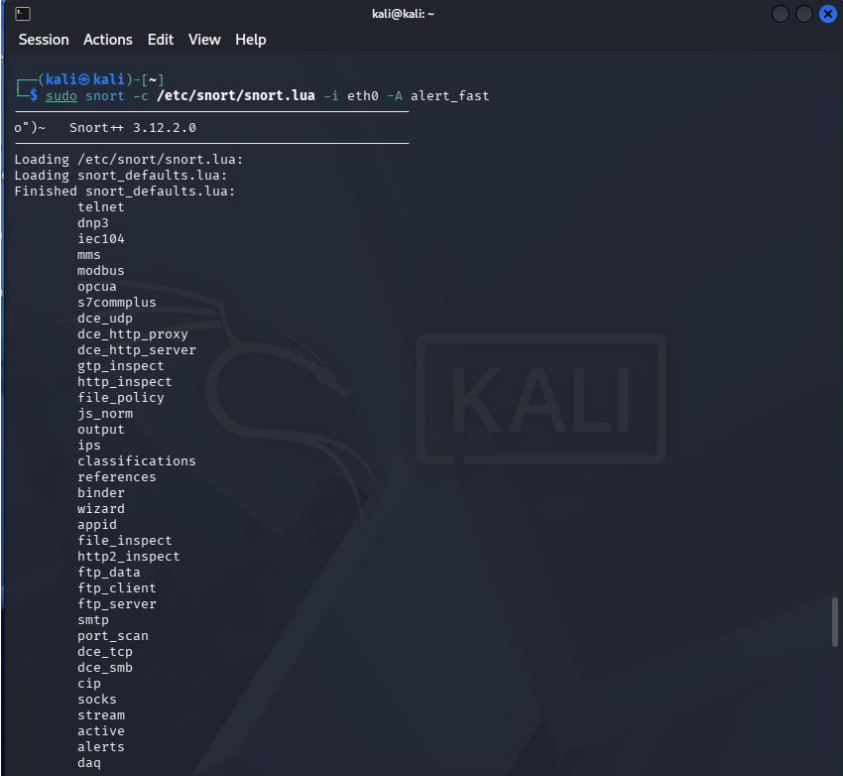
Exercise 1 – Port Scan Tests (Nmap)

```
# SYN scan (requires
root) sudo nmap -sS
192.168.1.1

# NULL scan

sudo nmap -sN 192.168.1.1
```

Week 6 Screenshot 1 – Snort Running in IDS Mode



```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo snort -c /etc/snort/snort.lua -i eth0 -A alert_fast
o")~ Snort++ 3.12.2.0
Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  telnet
  dnp3
  iec104
  mms
  modbus
  opcua
  s7commplus
  dce_udp
  dce_http_proxy
  dce_http_server
  gtp_inspect
  http_inspect
  file_policy
  js_norm
  output
  ips
  classifications
  references
  binder
  wizard
  appid
  file_inspect
  http2_inspect
  ftp_data
  ftp_client
  ftp_server
  smtp
  port_scan
  dce_tcp
  dce_smb
  cip
  socks
  stream
  active
  alerts
  daq
```

```

Finished ips.rules:
-----
ips policies rule stats
      id loaded shared enabled file
      1  223      0    223 /etc/snort/snort.lua

rule counts
  total rules loaded: 223
    text rules: 223
  option chains: 223
  chain headers: 3
-----
port rule counts
      tcp  udp  icmp  ip
  any      3    0    1    0
  total    3    0    1    0
-----
service rule counts
      file_id:  to-srv  to-cli
      total:    219    219
-----
fast pattern groups
      to_server: 1
      to_client: 1
-----
search engine (ac_bnfa)
      instances: 2
      patterns: 438
      pattern chars: 2602
      num states: 1832
      num match states: 392
      memory scale: KB
      total memory: 71.2812
      pattern memory: 19.6484
      match list memory: 28.4375
      transition memory: 22.9453
appid: MaxRss diff: 2840
appid: patterns loaded: 300
-----
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0

```

Week 6 Screenshot 1 – Snort running with “`sudo snort -c /etc/snort/snort.lua -i eth0 -A alert_fast`” command, showing active IDS mode listening on eth0

Exercise 2 – ICMP Ping Burst Test

```
# Send 50 rapid ICMP echo requests at 0.1-second intervals

ping -c 50 -i 0.1 192.168.1.1
```

Week 6 Screenshot 2 – Nmap Scan & Snort Alert Output

```

(kali@kali)-[~]
$ sudo nmap -sS 192.168.32.131
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-22 01:31 -0400
Nmap scan report for 192.168.32.131
Host is up (0.0025s latency).
Not shown: 976 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
34571/tcp open  unknown
MAC Address: 00:0C:29:EA:BF:D3 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.84 seconds

```

```

kali@kali: ~
Session Actions Edit View Help
kali@kali: ~ kali@kali: ~
search engine (ac_bnf)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 2840
appid: patterns loaded: 300

pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0
06/22-01:31:33.654688 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:443
06/22-01:31:33.654746 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:23
06/22-01:31:33.654794 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:8080
06/22-01:31:33.654848 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:445
06/22-01:31:33.654894 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:995
06/22-01:31:33.654954 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:139
06/22-01:31:33.654993 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:256
06/22-01:31:33.655043 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:3306
06/22-01:31:33.655084 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:1322
06/22-01:31:33.655130 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:6699
06/22-01:31:33.655130 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:425
06/22-01:31:33.655130 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:8994
06/22-01:31:33.655130 [**] [1:1000001:2] "PORTSCAN Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 19
2.168.32.135:35459 → 192.168.32.131:1334

```

Week 6 Screenshot 2 – Nmap Scan & Snort Alert Output

Exercise 3 – Suspicious HTTP Tests (curl)

```
# Simulate sqlmap user-agent

curl -A 'sqlmap/1.7' http://192.168.1.1/

# Simulate SQL injection in URL parameter

curl 'http://192.168.1.1/search?q=%27+OR+1%3D1'

# Simulate directory traversal

curl 'http://192.168.1.1/../../etc/passwd'
```

Week 6 Screenshot 3 – ICMP Burst & Ping Flood Alert

```
(kali@kali)-[~]
$ ping -c 50 -i 0.1 192.168.32.131
PING 192.168.32.131 (192.168.32.131) 56(84) bytes of data.
64 bytes from 192.168.32.131: icmp_seq=1 ttl=64 time=9.55 ms
64 bytes from 192.168.32.131: icmp_seq=2 ttl=64 time=0.611 ms
64 bytes from 192.168.32.131: icmp_seq=3 ttl=64 time=1.09 ms
64 bytes from 192.168.32.131: icmp_seq=4 ttl=64 time=0.358 ms
64 bytes from 192.168.32.131: icmp_seq=5 ttl=64 time=1.09 ms
64 bytes from 192.168.32.131: icmp_seq=6 ttl=64 time=0.346 ms
64 bytes from 192.168.32.131: icmp_seq=7 ttl=64 time=0.784 ms
64 bytes from 192.168.32.131: icmp_seq=8 ttl=64 time=1.40 ms
64 bytes from 192.168.32.131: icmp_seq=9 ttl=64 time=0.861 ms
64 bytes from 192.168.32.131: icmp_seq=10 ttl=64 time=0.996 ms
64 bytes from 192.168.32.131: icmp_seq=11 ttl=64 time=0.523 ms
64 bytes from 192.168.32.131: icmp_seq=12 ttl=64 time=1.06 ms
64 bytes from 192.168.32.131: icmp_seq=13 ttl=64 time=0.962 ms
64 bytes from 192.168.32.131: icmp_seq=14 ttl=64 time=0.566 ms
64 bytes from 192.168.32.131: icmp_seq=15 ttl=64 time=0.597 ms
64 bytes from 192.168.32.131: icmp_seq=16 ttl=64 time=0.411 ms
64 bytes from 192.168.32.131: icmp_seq=17 ttl=64 time=0.243 ms
64 bytes from 192.168.32.131: icmp_seq=18 ttl=64 time=2.11 ms
64 bytes from 192.168.32.131: icmp_seq=19 ttl=64 time=2.26 ms
64 bytes from 192.168.32.131: icmp_seq=20 ttl=64 time=1.74 ms
64 bytes from 192.168.32.131: icmp_seq=21 ttl=64 time=0.697 ms
64 bytes from 192.168.32.131: icmp_seq=22 ttl=64 time=0.942 ms
64 bytes from 192.168.32.131: icmp_seq=23 ttl=64 time=0.653 ms
64 bytes from 192.168.32.131: icmp_seq=24 ttl=64 time=0.493 ms
64 bytes from 192.168.32.131: icmp_seq=25 ttl=64 time=0.559 ms
64 bytes from 192.168.32.131: icmp_seq=26 ttl=64 time=0.817 ms
64 bytes from 192.168.32.131: icmp_seq=27 ttl=64 time=0.513 ms
64 bytes from 192.168.32.131: icmp_seq=28 ttl=64 time=0.863 ms
64 bytes from 192.168.32.131: icmp_seq=29 ttl=64 time=0.632 ms
64 bytes from 192.168.32.131: icmp_seq=30 ttl=64 time=0.747 ms
64 bytes from 192.168.32.131: icmp_seq=31 ttl=64 time=0.551 ms
64 bytes from 192.168.32.131: icmp_seq=32 ttl=64 time=0.295 ms
64 bytes from 192.168.32.131: icmp_seq=33 ttl=64 time=0.833 ms
64 bytes from 192.168.32.131: icmp_seq=34 ttl=64 time=0.516 ms
64 bytes from 192.168.32.131: icmp_seq=35 ttl=64 time=0.332 ms
```

```
pcap DAQ configured to passive.
Commencing packet processing
Retrv queue interval is: 200 ms
** [0] eth0
06/22-01:41:19.780058 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:19.884088 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:19.988004 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.092102 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.196084 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.301450 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.404130 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.508119 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.616082 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.723946 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.828056 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:20.932096 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:21.036443 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:21.143949 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:21.248171 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:21.359663 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
32.135 → 192.168.32.131
06/22-01:41:21.459591 [**] [1:1000004:2] "ICMP Ping Burst Detected" [**] [Priority: 0] {ICMP} 192.168
```

Week 6 Screenshot 3 – 'ping -c 50 -i 0.1' command output with Snort [**] ICMP Ping Burst Detected [**] alert firing in console

Week 7 – Alert Review & Findings Summary

Test Results Table

Test	Tool Used	Rule SID	Alert Fired?	Risk Level
SYN Port Scan	Nmap -sS	1000001	Yes	HIGH
NULL Scan	Nmap -sN	1000002	Yes	HIGH
XMAS Scan	Nmap -sX	1000003	Yes	HIGH
ICMP Ping Burst	ping -c 50	1000004	Yes	MEDIUM
sqlmap User-Agent	curl -A	1000005	Yes	HIGH
SQL Injection URI	curl	1000006	Yes	CRITICAL
Directory Traversal	curl	1000007	Yes	HIGH

Key Findings

- All 7 custom Snort rules fired correctly during controlled testing - 100% detection rate.
- detection_filter rules (port scan, ping burst) produced zero false positives during the baseline period.
- HTTP content-matching rules confirm Snort's deep packet inspection capability at Layer 7.
- Combining Snort alerts with Wireshark captures provided independent verification of each detection.
- Snort's console alert format provides timestamp, rule SID, rule message, protocol, and IP/port details.

Challenges & Solutions

Challenge	Solution Applied
snort.lua validation errors on first run	Used 'snort -T' flag to test config and identified missing rule-file path variable
Network interface not capturing traffic	Confirmed interface name with 'ip a'; corrected the -i flag value to eth0
detection_filter rules not triggering as expected	Reduced count detection_filter during testing; re-verified with repeated Nmap scans
HTTP content rules not matching	Enabled http_inspect preprocessor section in snort.lua
SID conflicts with existing Snort rules	Assigned unique SID range 1000001–1000099 reserved for all custom rules

Week 8 – Final Summary, Reflection & Conclusion

Master Snort Command Cheat Sheet

Tool	Command	Purpose
snort	sudo apt install snort -y	Install Snort on Kali Linux
snort	sudo snort -T -c /etc/snort/snort.lua	Validate configuration file
snort	sudo snort -c /etc/snort/snort.lua -i eth0 -A alert_fast	Run IDS mode with console alerts
nmap	sudo nmap -sS	SYN port scan
nmap	sudo nmap -sN	NULL scan
nmap	sudo nmap -sX	XMAS scan
ping	ping -c 50 -i 0.1	ICMP burst test (50 packets)
curl	curl -A 'sqlmap/1.7' http://	Simulate sqlmap user-agent
curl	curl -I http://	Fetch HTTP response headers only
wireshark	wireshark &	Launch GUI for packet capture

Project Findings Summary

Finding	Tool Used	Risk Level	Details
7/7 custom rules triggered	Snort + Nmap/ping/curl	-	All planned test cases triggered the intended Snort rules during controlled laboratory testing
Port scan detected	Snort + Nmap	HIGH	SYN, NULL, XMAS scans all alerted
ICMP flood detected	Snort + ping	MEDIUM	Burst of 10+ pings/5s triggered alert
HTTP attacks detected	Snort + curl	CRITICAL	sqlmap UA, SQLi, dir traversal all caught
detection_filter logic verified	Snort + Wireshark	-	No false positives on baseline traffic

Conclusion

This project provided comprehensive hands-on experience with network intrusion detection using Snort on a Kali Linux virtual machine across eight structured weeks. Starting from core IDS concepts, the project progressed through installation, baseline analysis, rule authoring, live testing, and documentation - mirroring a real penetration testing engagement workflow in a safe, isolated lab environment.

The most significant lesson was that effective IDS deployment requires *both* careful rule design (to minimize false positives) and thorough baseline traffic analysis (to know what is normal). Snort's rule language enables precise packet matching at the network, transport, and application layers - making it a flexible and powerful detection tool.

Future work could include integrating Snort 3 with ELK Stack or Wazuh for centralized monitoring and alert management.

References

- Snort Official Documentation - <https://www.snort.org/documents>
- Kali Linux Documentation - <https://www.kali.org/docs/>
- Wireshark User Guide - https://www.wireshark.org/docs/wsug_html/
- Nmap Network Scanning (Gordon Lyon) - <https://nmap.org/book/>
- Snort 3 Rule Writing Guide - <https://docs.snort.org/rules/>